

Chapter 5: Router Pattern (Intent Classification, Capability Routing, and Dynamic Agent Selection)

Part II: Agentic Architecture Patterns

Book: Agentic AI & RAG Systems Handbook

Version: 1.0.0

Chapter ID: 05

File Name: chapter-05-router-pattern.md

Chapter Overview

As agent ecosystems grow, a fundamental challenge emerges:

Which agent, tool, workflow, model, or execution path should handle a request?

The Router Pattern solves this problem.

Without routing:

```
1 User
2 ↓
3 Single Agent
```

With routing:

```
1 User
2 ↓
3 Router
4 ↓
5 Best Resource
```

Routers are among the most important architectural components in enterprise AI because they determine:

- Which model is used
- Which agent is selected
- Which workflow executes
- Which tools become available
- Which approval path is required

Nearly every production multi-agent platform, enterprise copilot, RAG assistant, and digital workforce implementation relies on routing.

Learning Objectives

After completing this chapter you should be able to:

- Explain the Router Pattern
 - Build intent classification systems
 - Design capability-based routing
 - Implement multi-agent routing architectures
 - Compare rule-based and LLM routing
 - Build routing systems in LangGraph
 - Understand routing observability
 - Design enterprise-grade routing platforms
 - Avoid routing anti-patterns
 - Discuss router architectures in principal-level interviews
-

Prerequisites

Required:

- Chapter 1: Generative AI Foundations
 - Chapter 2: Agentic AI Fundamentals
 - Chapter 3: Agent Design Principles
 - Chapter 4: ReAct Pattern
-

1. Introduction

Consider an enterprise copilot with:

- HR Agent
- Legal Agent
- Finance Agent
- Security Agent
- DevOps Agent
- Compliance Agent

User asks:

1 Show current PTO policy.

Only the HR Agent should handle this.

Another request:

1 Review PCI compliance requirements.

Compliance Agent should handle it.

Without routing:

1 Every Agent
2 Attempts Everything

This quickly becomes:

1 Slow
2 Expensive
3 Inaccurate
4 Ungovernable

The Router Pattern introduces specialization.

2. Core Concepts

What Is A Router?

Definition

A router is a decision-making component that selects the most appropriate execution path based on request characteristics.

Typical outputs:

- Agent
 - Tool
 - Workflow
 - Model
 - Knowledge Source
 - Human Approval Path
-

Core Routing Principle

1 Right Request
2 ↓

3	Right Resource
4	↓
5	Right Outcome

3. Why Routers Exist

As systems scale:

1	Users
2	↑
3	Requests
4	↑
5	Agents
6	↑
7	Tools
8	↑
9	Workflows

Complexity increases exponentially.

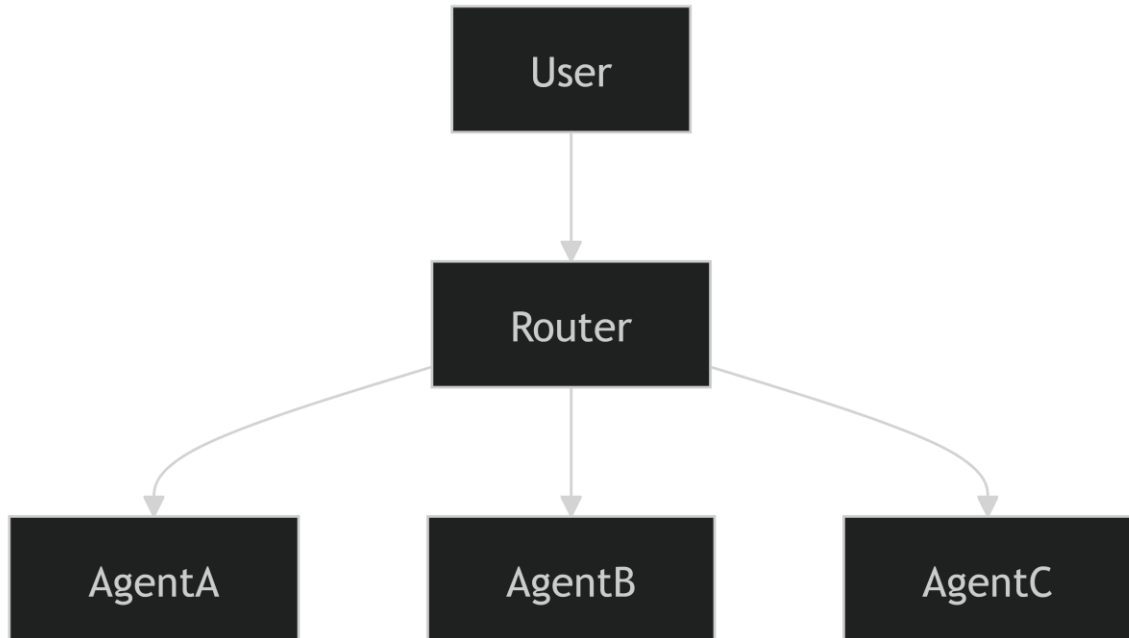
The router becomes the control point.

Benefits:

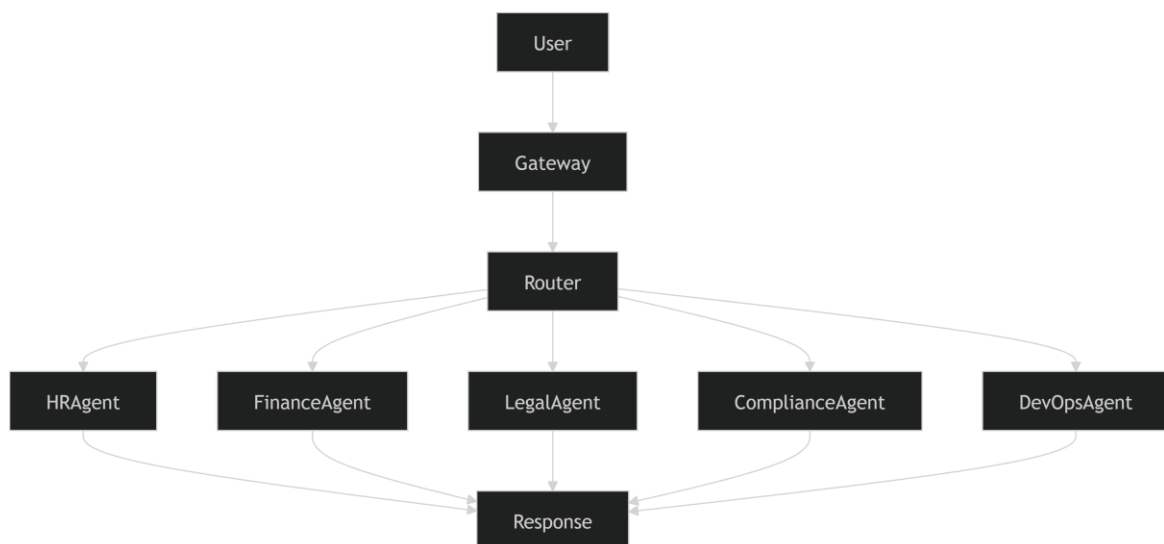
- Specialization
 - Cost reduction
 - Better governance
 - Better accuracy
 - Reduced latency
-

4. Router Architecture

Basic Router



Enterprise Router



5. Intent Classification

Intent classification is the most common routing mechanism.

Example

Input:

1 Reset my password.

Intent:

1 IT Support

Another Example

Input:

1 What is the reimbursement policy?

Intent:

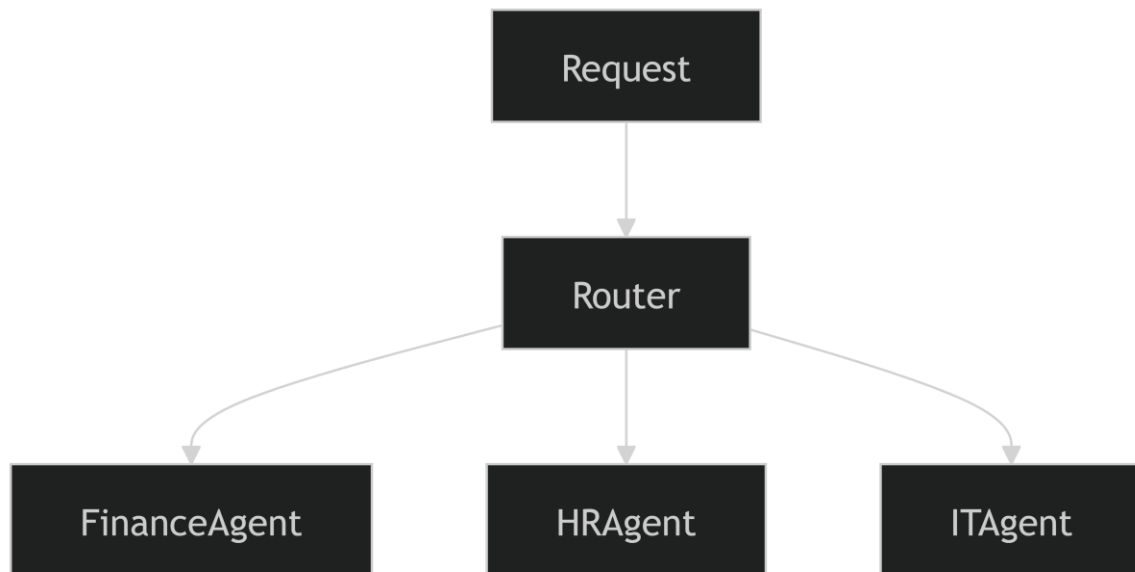
1 HR

Classification Flow



6. Single-Level Routing

Simple architecture:

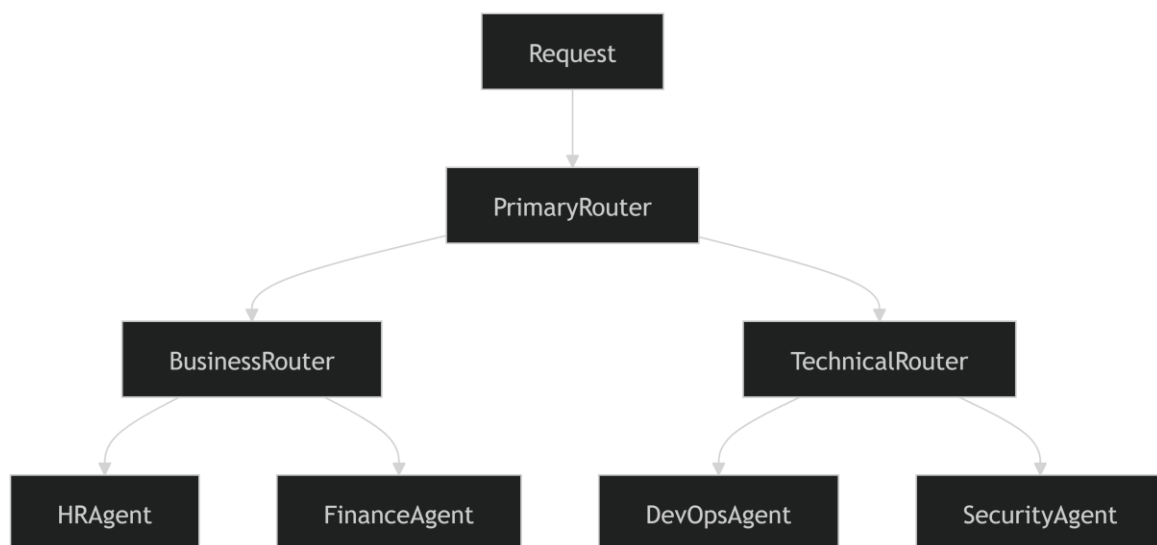


Suitable for:

- Small copilot systems
 - Department assistants
 - Internal support bots
-

7. Multi-Level Routing

Large enterprises frequently require hierarchical routing.



Benefits:

- Scalability
 - Better separation of concerns
 - Lower model complexity
-

8. Capability-Based Routing

Preferred enterprise pattern.

Route by capability instead of agent name.

Bad Design

1 Send request to FinanceAgent

Better Design

1 Need capability:
2 financial_analysis

Registry resolves:

1 FinanceAgent

Architecture

Request



CapabilityRouter



Registry



Agent

9. Routing Strategies

Strategy 1: Rule-Based Routing

Example:

```
1 if "vacation" in query:
2     return "HRAgent"
```

Advantages

- Fast
- Explainable
- Cheap

Disadvantages

- Difficult to scale
 - Brittle
-

Strategy 2: LLM Routing

Example:

```
1 Classify request category.
```

LLM returns:

```
1 Compliance
```

Advantages

- Flexible
- High accuracy

Disadvantages

- Cost
 - Latency
 - Probabilistic
-

Strategy 3: Embedding-Based Routing

Convert request into embeddings.

Match against:

1 Agent Capability Embeddings

Select most similar capability.

Advantages

- Scalable
- Dynamic

Disadvantages

- Requires embedding maintenance
-

Strategy 4: Hybrid Routing

Enterprise recommendation:

1	Rules
2	+
3	Embeddings
4	+
5	LLM Verification

10. Risk-Based Routing

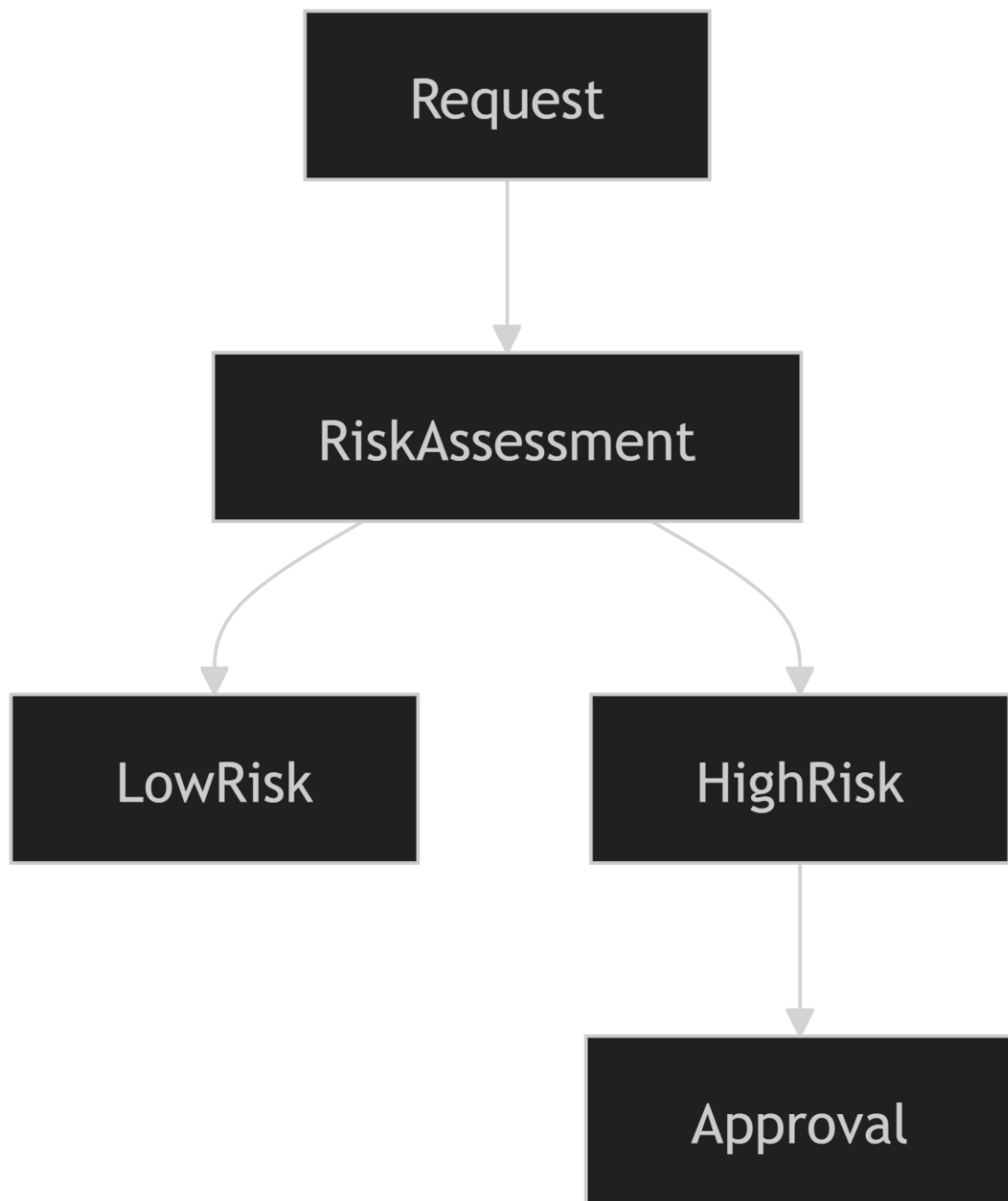
Certain requests require specialized handling.

Example:

1 Delete customer records.

Route:

1 High Risk Workflow



11. Complexity Routing

Different requests may deserve different models.

Simple Request

1 Summarize email.

Route:

1 Small Model

Complex Request

1 Perform compliance impact analysis.

Route:

1 Large Reasoning Model

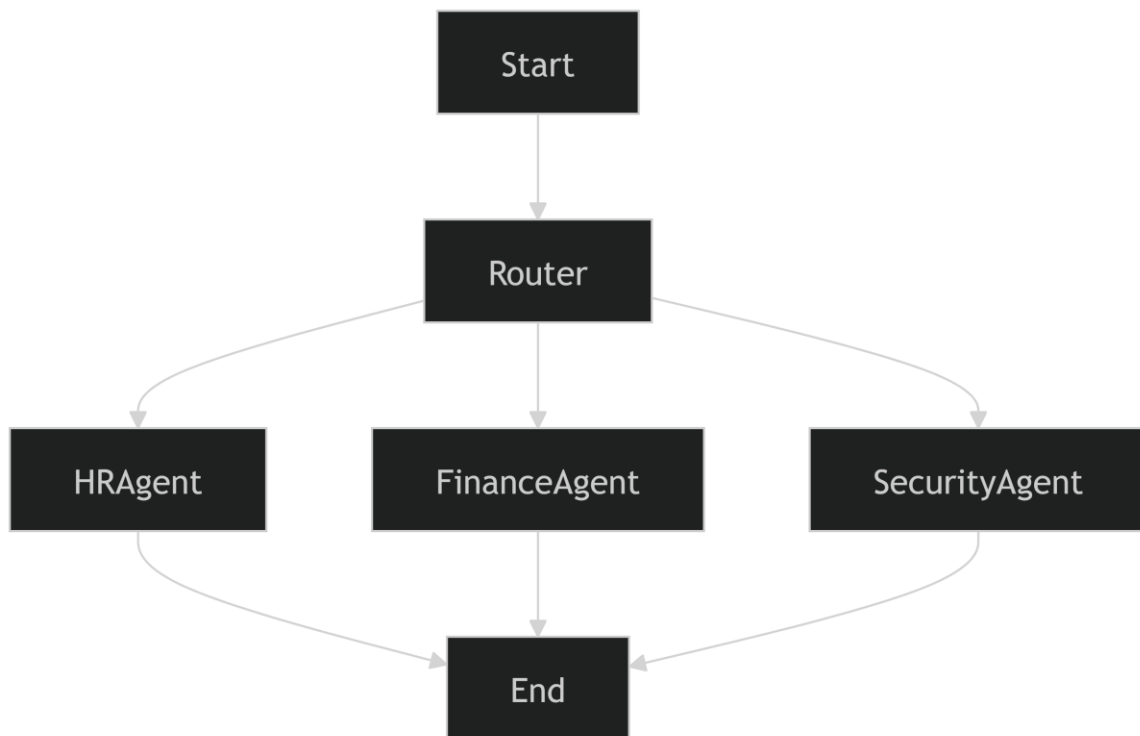
Benefits:

- Cost reduction
 - Efficiency
-

12. LangGraph Router Pattern

LangGraph excels at routing.

Graph



State

```
1 from typing import TypedDict
2
3 class RouterState(TypedDict):
4     question: str
5     route: str
```

Router Node

```
1 def route(state):
2
3     q = state["question"]
4
5     if "salary" in q:
6         return {"route": "finance"}
7
8     return {"route": "general"}
```

Conditional Edge

```
1 graph.add_conditional_edges(  
2     "router",  
3     routing_function  
4 )
```

13. CrewAI Router Pattern

CrewAI often uses manager agents.

```
1 Manager  
2 ↓  
3 Choose Worker  
4 ↓  
5 Execute
```

Equivalent to routing.

14. AutoGen Router Pattern

AutoGen frequently routes conversation tasks between agents.

```
1 Coordinator  
2 ↓  
3 Agent  
4 ↓  
5 Coordinator
```

15. Semantic Kernel Router Pattern

Semantic Kernel planners often determine:

```
1 Which skill?  
2 Which plugin?  
3 Which function?
```

This is routing.

16. OpenAI Router Pattern

OpenAI systems commonly route:

- Models
- Functions
- Workflows

based on prompts or classifiers.

17. Production Example

Enterprise Copilot

Departments:

1	HR
2	Finance
3	Legal
4	Security
5	IT

Request

1	How many vacation days do I get?
---	----------------------------------

Router

1	HR
---	----

Execution

1	HR Agent
---	----------

Response

1	Policy Answer
---	---------------

18. Cloud Architecture

AWS

1	API Gateway
2	↓
3	Router Lambda
4	↓
5	Agent Services

Services:

- Bedrock
 - Lambda
 - EventBridge
 - ECS/EKS
-

Azure

1	Azure OpenAI
2	↓
3	Router Service
4	↓
5	Specialized Agents

GCP

1	Vertex AI
2	↓
3	Router
4	↓
5	Agent Platform

19. Scalability

Router scalability is often overlooked.

Small Environment

1	<10 agents
---	------------

Simple routing.

Large Environment

1 100+ agents

Requires:

- Registries
 - Discovery
 - Capability metadata
 - Caching
-

20. Reliability

Routers require:

- Retry policies
- Fallback routes
- Auditability
- Confidence thresholds

Example:

1 Confidence < 0.70
2 ↓
3 Human Escalation

21. Security

Routing is a security boundary.

Questions:

1 Which agent can receive this request?
2 Which permissions apply?
3 Which tools become available?

Security Model

Request



Router



Policy



AuthorizedAgent

22. Observability

Track:

Route Accuracy

1 Correct Routes

Route Latency

1 Time to Select Path

Misroutes

1 Incorrect Agent

Escalations

1 Fallback Routes

23. Common Mistakes

Mistake 1

Routing directly to tools.

Mistake 2

No capability registry.

Mistake 3

Using only LLM classification.

Mistake 4

Ignoring confidence scores.

Mistake 5

No failure handling.

24. Anti-Patterns

Micro-Agent Explosion

1 500 tiny agents

Impossible to manage.

Routing Loops

1 Agent A
2 ↓
3 Router
4 ↓
5 Agent B
6 ↓
7 Router
8 ↓
9 Agent A

Infinite cycle.

Hardcoded Routing

1 if query:
2 send_to_agent()

Poor scalability.

Direct Tool Routing

Bypasses governance.

25. Architecture Decision Matrix

Scenario	Router Fit
Enterprise Copilot	Excellent
Multi-Agent System	Essential
Department Assistant	Excellent
Compliance Platform	Excellent
Small FAQ Bot	Limited
Static Workflow	Low Need

26. Real-World Use Cases

Enterprise Copilot

Department routing.

Security Operations

Specialized workflow routing.

Compliance Platform

Risk-based routing.

Customer Support

Intent routing.

Agentic Operations Platform

Capability routing.

27. Interview Questions

Q1

What is the Router Pattern?

Q2

Why does routing matter?

Q3

What is capability-based routing?

Q4

What is the best enterprise routing strategy?

Q5

How do routers improve governance?

Q6

What are common routing failures?

Q7

When should hierarchical routing be used?

Q8

How does LangGraph implement routing?

28. Interview Answers

Q1

A decision layer that selects the optimal execution path, agent, workflow, or tool.

Q2

It enables specialization, scalability, governance, and cost control.

Q3

Routing based on capabilities rather than agent names.

Q4

Hybrid routing combining rules, embeddings, and LLM evaluation.

Q5

It limits access to appropriate agents and tools.

Q6

Misclassification, loops, hardcoded dependencies, and absent fallback routes.

Q7

When organizations have many domains, agents, or capabilities.

Q8

Using conditional edges and state-based branching.

29. Architecture Review Checklist

Router Readiness

- Intent model defined
 - Capability catalog defined
 - Registry implemented
 - Routing confidence tracked
 - Fallback path defined
 - Security reviewed
 - Governance applied
 - Observability configured
 - Escalation mechanism implemented
-

30. Practical Exercises

1. Design a router for an enterprise copilot.
 2. Compare rules-based and LLM routing.
 3. Build a capability registry.
 4. Implement LangGraph conditional routing.
 5. Create a risk-based routing architecture.
-

31. Key Takeaways

- Routers are the traffic controllers of agent ecosystems.
- Routing enables specialization and scalability.
- Capability-based routing is preferred over agent-name routing.
- Hybrid routing strategies generally outperform single approaches.
- Routing is both an architecture concern and a security concern.
- LangGraph provides first-class routing support.
- Large enterprises require hierarchical routing strategies.
- Observability and confidence scoring are critical.

- Poor routing architecture creates bottlenecks and governance failures.
 - Nearly every production multi-agent system relies on routing.
-

32. References

- [LangGraph Documentation](#)
 - [CrewAI Documentation](#)
 - [AutoGen Documentation](#)
 - [Semantic Kernel Documentation](#)
 - [OpenAI Function Calling Documentation](#)
 - [Enterprise Agent Architecture Patterns](#)
 - [Agentic AI Design Best Practices](#)
-

Chapter Status: Completed

Markdown Ready: Yes

PDF Ready: Yes (pending build pipeline)

Next Chapter: Chapter 6 – Planner-Executor Pattern and Hierarchical Task Decomposition